



Figure 1: How Wasm works

How WebAssembly fits in a Web platform

The Web platform can be thought of as having two parts:

- A virtual machine (VM) that runs the Web app's code, e.g., the JavaScript code that powers your apps.
- A set of Web APIs that the Web app can call to control browser and device functionality, and that makes things happen (DOM, CSSOM, WebGL, IndexedDB, Web Audio API, etc).

Historically, the VM has been able to load only JavaScript. This has worked well for us as JavaScript is powerful enough to solve most problems people have on the Web today. We have run into performance problems, however, when trying to use JavaScript for 3D games, virtual and augmented reality, computer vision, image or video editing, and a number of other domains that demand native performance.

WebAssembly is a language that is different from JavaScript, but it is not intended as a replacement. Instead, it is designed to complement and work alongside JavaScript, allowing Web developers to take advantage of both languages' strong points.

- JavaScript is a high-level language that is flexible and expressive enough to write Web applications. It has many advantages — it is dynamically typed, requires no compile steps, and has a huge ecosystem that provides powerful frameworks, libraries and other tools.
- WebAssembly is a low-level Assembly-like language with a compact binary format that runs with near-native performance. It provides languages with low-level memory models such as C++ and Rust with a compilation target so that they can run on the Web.

With the advent of WebAssembly in browsers, they can now load and run two types of code — JavaScript as well as WebAssembly.

The WebAssembly dictionary

There are several keywords that users need to be aware of to understand how WebAssembly runs in the browser. These are mentioned below.

- **Module:** This represents a WebAssembly binary that has been compiled by the browser into executable machine code. A module is stateless and thus, like a blob, can be explicitly shared between windows and workers (via `postMessage()`). A module declares imports and exports just like an ES2015 module.
- **Memory:** This is a resizable array buffer that contains the linear array of bytes read and written by WebAssembly's

low-level memory access instructions.

- **Table:** This is a resizable typed array of references (e.g., to functions) that cannot otherwise be stored as raw bytes in memory (for safety and portability reasons).
- **Instance:** This is a module paired with all the state it uses at runtime, including a memory, table, and a set of imported values.

The JavaScript API provides developers with the ability to create modules, memories, tables and instances. Given a WebAssembly instance, JavaScript code can synchronously call its exports, which are exposed as normal JavaScript functions.

Let us now look at an example that has been broken down into three steps.

Step 1: Writing C code

We are writing a simple C program that picks the random numbers, 1 and 6. Let's make a new file in a working directory of your choice and name it *dice-roll.c*.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <emscripten/emscripten.h>
int main(int argc, char ** argv) {
    printf("WebAssembly module loaded\n");
}

int EMSCRIPTEN_KEEPALIVE roll_dice() {
    srand ( time(NULL) );
    return rand() % 6 + 1;
}
  
```

When we compile the Wasm file and load this code into the browser, the function `main()` will be automatically executed. The contents of `printf()` will be displayed in the `console.log` when executed. We need the `roll_dice` function to be available in JavaScript and let the Emscripten compiler know about it by adding `EMSCRIPTEN_KEEPALIVE`. The Emscripten compiler is a source-to-source compiler that runs as a back end. It is used in this case because the Emscripten tool is able to take any C/C++ source code and compile it into a `.wasm` module, while loading the necessary JavaScript code for running the module as well as an HTML file to display the results of the code.

Step 2: Compiling C to WebAssembly

Now that we have our simple C program, we need to compile it into Wasm. Not only that, we also need to generate JavaScript glue code that will help us actually run it.

Here we have to use the Emscripten compiler. There are tons of CLI options available and many different approaches. I found the following combination most user friendly:

```
emcc dice-roll.c -s WASM=1 -O3 -o index.js
```